Analysis of Algorithms: time & space

Dr. Jeyakesavan Veerasamy

jeyv@utdallas.edu

The University of Texas at Dallas, USA

Program running time

When is the running time (waiting time for user) noticeable/important?

Program running time – Why?

When is the running time (waiting time for user) noticeable/important?

- web search
- database search
- real-time systems with time constraints

Factors that determine running time of a program

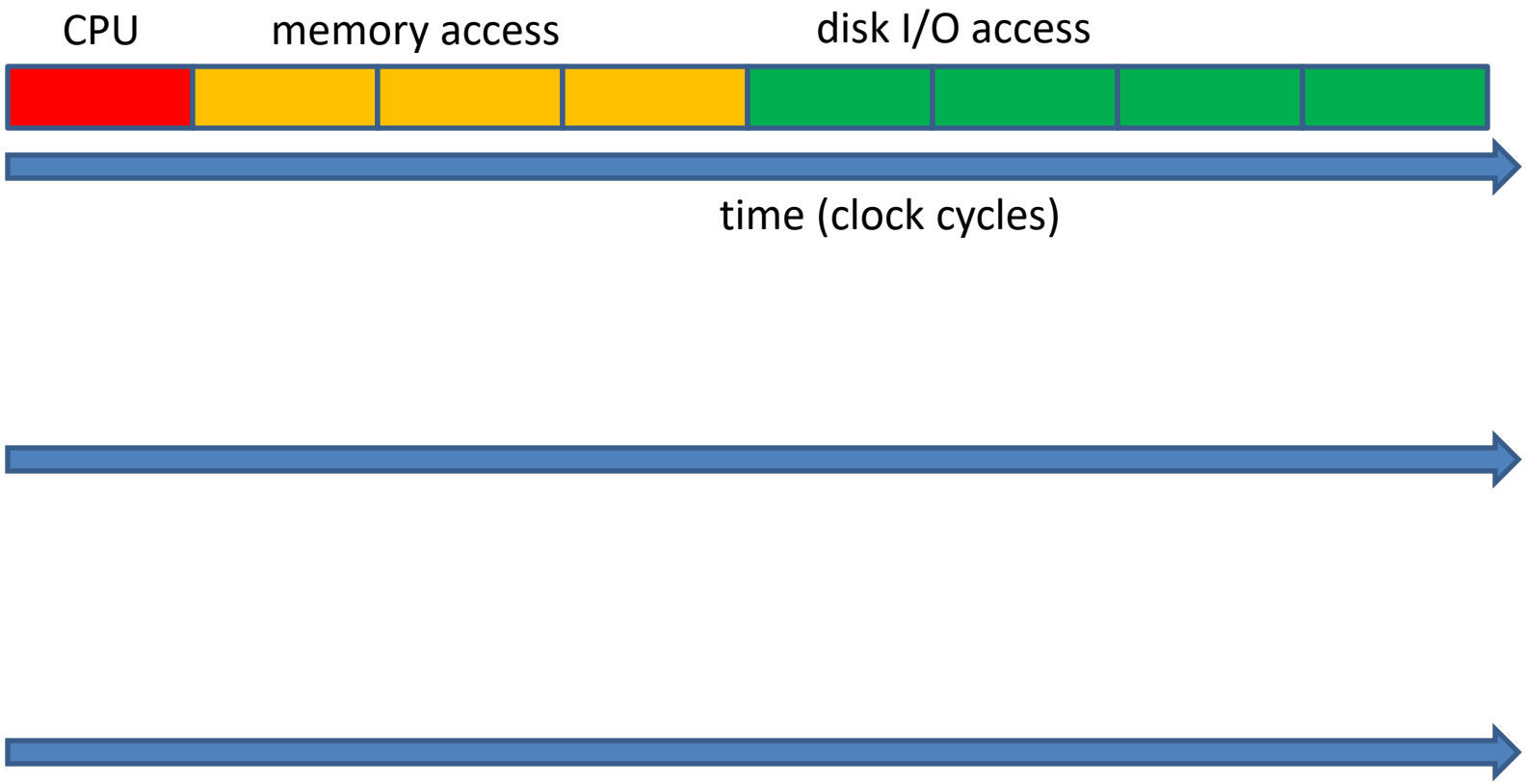
Factors that determine running time of a program

- problem size: n
- basic algorithm / actual processing
- memory access speed
- CPU/processor speed
- # of processors?
- compiler/linker optimization?

Running time of a program or transaction processing time

- amount of input: n → min. linear increase
- basic algorithm / actual processing → depends on algorithm!
- memory access speed → by a factor
- CPU/processor speed → by a factor
- # of processors? → yes, if multi-threading or multiple processes are used.
- compiler/linker optimization? → ~20%

Running time for a program: a closer look



Time Complexity

- measure of algorithm efficiency
- has a big impact on running time.
- Big-O notation is used.
- To deal with n items, time complexity can be $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, even $O(n^n)$.

Coding example #1

```
for ( i=0 ; i<n ; i++ )
    m += i;
```

Coding example #2

```
for ( i=0 ; i<n ; i++ )
        for( j=0 ; j<n ; j++ )
              sum[i] += entry[i][j];
```

Coding example #3

```
for ( i=0 ; i<n ; i++ )
        for( j=0 ; j<i ; j++ )
            m += j;
```

Coding example #4

```
i = 1;
while (i < n) {
     tot += i;
     i = i * 2;
}
```

Example #4: equivalent # of steps?

```
i = n;
while (i > 0) {
     tot += i;
     i = i / 2;
}
```

Coding example #5

```
for ( i=0 ; i<n ; i++ )
    for( j=0 ; j<n ; j++ )
        for( k=0 ; k<n ; k++ )
            sum[i][j] += entry[i][j][k];
```

Coding example #6

```
for ( i=0 ; i<n ; i++ )
        for( j=0 ; j<n ; j++ )
                sum[i] += entry[i][j][0];

for ( i=0 ; i<n ; i++ )
        for( k=0 ; k<n ; k++ )
                sum[i] += entry[i][0][k];
```

Coding example #7

```
for ( i=0 ; i<n ; i++ )
        for( j=0 ; j< sqrt(n) ; j++ )
            m += j;
```

Coding example #8

```
for ( i=0 ; i<n ; i++ )
        for( j=0 ; j< sqrt(995) ; j++ )
            m += j;
```

Coding example #8 : Equivalent code

```
for ( i=0 ; i<n ; i++ )
{
      m += j;
      m += j;
      m += j;
      …
      m += j;    // 31 times
}
```

Coding example #9

```
int total(int n)
    for( i=0 ; i < n; i++)
      subtotal += i;

main()
    for ( i=0 ; i<n ; i++ )
      tot += total(i);
```

Coding example #9: Equivalent code

```
for ( i=0 ; i<n ; i++ ) {
   subtotal = 0;
   for( j=0 ; j < i; j++)
       subtotal += j;
   tot += subtotal;
 }
```

Compare running time growth rates

n	$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$
10^2	1 μsec	1 μsec	1 μsec	1 μsec	1 μsec
10^3	1 μsec	1.5 μsec	10 μsec	15 μsec	100 μsec
10^4	1 μsec	2 μsec	100 μsec	200 μsec	10 msec
10^5	1 μsec	2.5 μsec	1 msec	2.5 msec	1 sec
10^6	1 μsec	3 μsec	10 msec	30 msec	1.7 min
10^7	1 μsec	3.5 μsec	100 msec	350 msec	2.8 hr
10^8	1 μsec	4 μsec	1 sec	4 sec	11.7 d

n	$O(n^2)$	$O(2^n)$
100	1 μsec	1 μsec
110	1.2 μsec	1 msec
120	1.4 μsec	1 sec
130	1.7 μsec	18 min
140	2.0 μsec	13 d
150	2.3 μsec	37 yr
160	2.6 μsec	37,000 yr

Time Complexity → maximum N?

complexity	maximum N
$\Theta(N)$	100 000 000
$\Theta(N \log N)$	40 000 000
$\Theta(N^2)$	10 000
$\Theta(N^3)$	500
$\Theta(N^4)$	90
$\Theta(2^N)$	20
$\Theta(N!)$	11

Table 3. Approximate maximum problem size solvable in 8 seconds.

http://www.topcoder.com/tc?module=Static&d1=tutorials&d2=complexity1

Practical Examples

Example #1: carry n items from one room to another room

Example #1: carry n items from one room to another room

- How many operations?
- n pick-ups, n forward moves, n drops and n reverse moves → 4 n operations
- 4n operations = c. n = O(c. n) = O(n)
- Similarly, any program that reads n inputs from the user will have minimum time complexity O(n).

Example #2: Locating patient record in Doctor Office

What is the time complexity of search?

Example #2: Locating patient record in Doctor Office

What is the time complexity of search?

- Binary Search algorithm at work
- O(log n)
- Sequential search?
- O(n)

Example #3: Store manager gives gifts to first 10 customers

- There are n customers in the queue.
- Manager brings one gift at a time.

Example #3: Store manager gives gifts to first 10 customers

- There are n customers in the queue.
- Manager brings one gift at a time.
- Time complexity = O(c. 10) = O(1)
- Manager will take exactly same time irrespective of the line length.

Example #4: Thief visits a Doctor with Back Pain

Example #4: Thief visits a Doctor with Back Pain

- Doctor asks a few questions:
  - Is there a lot of stress on the job?
  - Do you carry heavy weight?

Example #4: Thief visits a Doctor with Back Pain

- Doctor asks a few questions:
  - Is there a lot of stress on the job?
  - Do you carry heavy weight?
- Doctor says: Never carry > 50 kgs

Knapsack problems

- Item weights: 40, 10, 46, 23, 22, 16, 27, 6
- Instance #1: Target : 50

- Instance #2: Target: 60

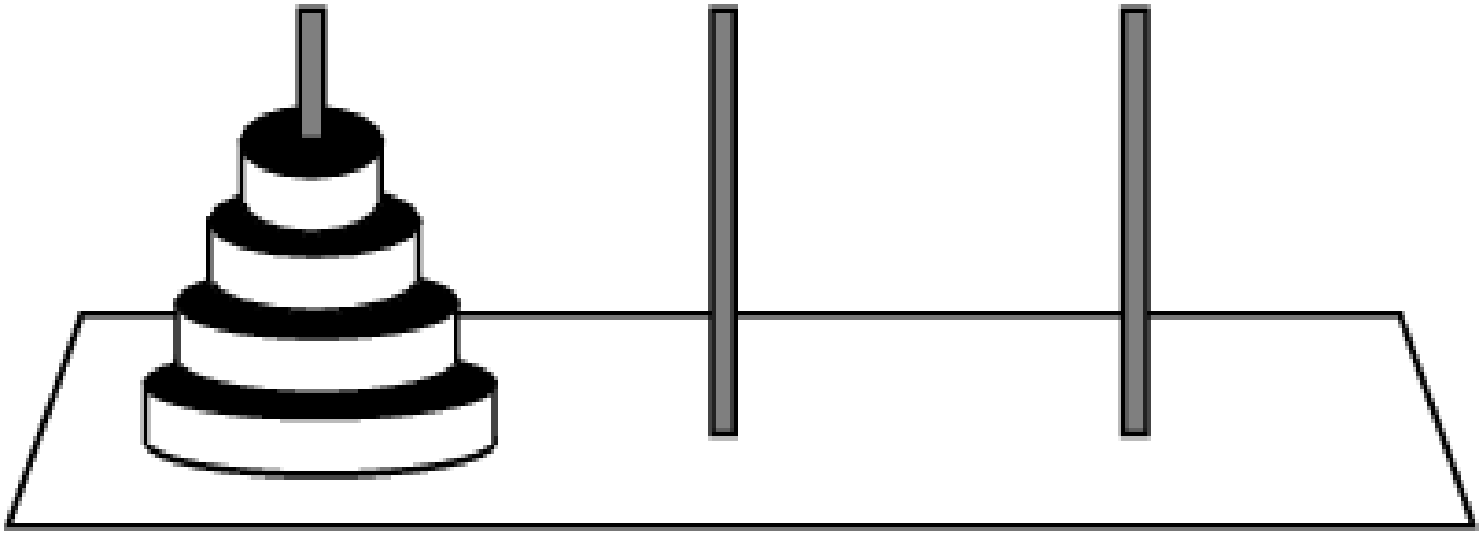
- Instance #3: Target: 70

Knapsack problem : Simple algorithm

Knapsack problem : Greedy algorithm

Knapsack problem : Perfect algorithm

Example #5: Hanoi Towers



Hanoi Towers: time complexity

Hanoi Towers: n pegs?

Hanoi Towers: (log n) pegs?

A few practical scenarios

Game console

- Algorithm takes longer to run → requires higher-end CPU to avoid delay to show output & keep realism.

Web server

- Consider 2 web-server algorithms: one takes 5 seconds & another takes 20 seconds.

Database access

Since the database load & save operations take O(n), why bother to optimize database search operation?

Daily data crunching

- Applicable for any industry that collects lot of data every day.
- Typically takes couple of hours to process.
- What if it takes >1 day?

Data crunching pseudocode

- initial setup
- loop
  - read one tuple
  - open db connection
  - send request to db
  - get response from db
  - close db
- post-processing

Data crunching pseudocode

- initial setup
- loop
  - read one tuple
  - open db connection
  - send request to db
  - get response from db
  - close db
- post-processing

- Equation for running time = $c_1 . n + d_1$
- Time complexity is O(n)

Data crunching pseudocode

- initial setup
- open db connection
- loop
  - read one tuple
  - send request to db
  - get response from db
- close db
- post-processing

- Equation for running time = $c_2 . n + d_2$
- Time complexity is still O(n), but the constants are different.
- $c_2 < c_1$
- $d_2 > d_1$

Search algorithms

- Sequential search
- Binary search
- Hashing

Summary

- Time complexity is a measure of algorithm efficiency
- Efficient algorithm plays the major role in determining the running time.

Q: Is it possible to determine running time based on algorithm's time complexity alone?

- Minor tweaks in the code can cut down the running time by a factor too.
- Other items like CPU speed, memory speed, device I/O speed can help as well.
- For certain problems, it is possible to allocate additional space & improve time complexity.

Questions & Answers

jeyv@utdallas.edu